MANUAL TÉCNICO E OPERACIONAL

Casa Mamãe Margarida

Documentação consolidada do site, painel de gestão, área da usuária, banco de dados, segurança e publicação.

Gerado em 12 de novembro de 2026

Os estados e resultados de serviços externos descritos nos guias são registros da data indicada em cada documento e devem ser conferidos antes de executar procedimentos.

Conteúdo

Visão geral do projeto

Manual de operação e manutenção

Área da usuária

Integração com o Supabase

Este PDF reúne a visão geral do projeto, o manual de operação, a documentação da área da usuária e as instruções de integração Supabase.

Visão geral do projeto

Visão geral

O projeto é o site institucional e a plataforma de gestão da Casa Mamãe Margarida. Reúne páginas públicas, painel da equipe e área individual da usuária. É uma aplicação multipágina feita com HTML, CSS e JavaScript modular, compilada pelo Vite. Supabase fornece autenticação, banco PostgreSQL, políticas de acesso (RLS) e armazenamento de arquivos.

Começar a trabalhar

Requisitos: Node.js 22.12 ou superior e npm.

```
npm.cmd install
npm.cmd run dev
```

Abra a URL local informada pelo Vite. Não abra os HTML diretamente com `file://`, pois as páginas usam módulos JavaScript.

Comandos disponíveis:

Comando	Uso
`npm run dev`	Inicia o servidor local do Vite.
`npm run build`	Gera as páginas prontas para publicação em `dist/`.
`npm run preview`	Serve localmente a compilação de `dist/`.
`npm run docs:pdf`	Regera o PDF consolidado da documentação.
`npm test`	Executa testes unitários, de validação e segurança com banco PostgreSQL local simulado.
`npm run test:e2e`	Executa os testes de navegador Playwright.
`npm run test:supabase`	Faz verificações de leitura na conexão Supabase configurada.

No PowerShell do Windows, os comandos podem ser executados como `npm.cmd`. O diagnóstico Supabase pode aceitar `-- --insert-test`, que cria um registro sintético; use essa opção somente quando quiser explicitamente gravar um dado de teste.

Páginas

Site público

- `index.html`: apresentação, cursos e projetos publicados.
- `voluntariado.html`: informações, oportunidades e formulário de voluntariado.

- `solicitar-atendimento.html`: solicitação pública de atendimento.
- `doacoes.html`: informações e registro de intenção de doação; não processa pagamentos.
- `contato.html`: formulário de contato.

Acesso e área individual

- `login.html`, `recuperar-senha.html` e `redefinir-senha.html`: autenticação e recuperação de acesso.
- `cadastro.html` e `area-usuario.html`: criação de conta e página inicial da usuária.
- `minhas-consultas.html`, `agenda.html`, `minhas-solicitacoes.html`, `historico.html`, `perfil.html` e `notificacoes.html`: acompanhamento individual.
- `nova-solicitacao.html`: solicitação de atendimento pela área autenticada.

Painel da equipe

- `dashboard.html`: indicadores.
- `beneficiarias.html` e `atendimentos.html`: registros e encontros de atendimento.
- `cursos.html` (e a rota legada `servicos.html`): cadastro e publicação de cursos.
- `projetos.html`: cadastro e publicação de projetos.
- `solicitacoes.html`: solicitações feitas pela área da usuária.
- `profissionais.html`: profissionais responsáveis pelos atendimentos.
- `gestao-doacoes.html`, `mensagens.html`, `relatorios.html`, `usuarios.html` e `configuracoes.html`: gestão administrativa.

O Vite declara as páginas de entrada em `vite.config.js`. Ao adicionar uma página HTML, inclua-a nessa lista para que ela faça parte da compilação multipágina.

Organização do código

Caminho	Responsabilidade
`css/style.css`	Estilos compartilhados e site público, incluindo navegação e layout responsivo.
`css/dashboard.css`	Painel administrativo.
`css/usuario.css`	Área individual da usuária.
`css/login.css`	Login e recuperação de senha.
`css/voluntariado.css`	Estilos próprios da página de voluntariado.
`js/layout.js`	Monta cabeçalho, navegação e rodapé públicos.
`js/main.js`	Inicialização da página principal, catálogo e interações públicas.
`js/forms.js`	Formulários públicos.
`js/voluntariado.js`	Interações da página de voluntariado.
`js/supabase.js`	Inicializa o cliente Supabase e a persistência de sessão.

Caminho	Responsabilidade
`js/login.js`, `js/cadastro.js`	Autenticação, criação de conta e recuperação de acesso.
`js/dashboard.js`	Painel, operações de dados, tabelas, formulários e relatórios.
`js/admin-schema.js`	Colunas, campos e opções de cadastro do painel.
`js/usuario.js`, `js/usuario-data.js`	Páginas e dados da área individual.
`js/auth-routing.js`	Direcionamento após autenticação.
`js/utils.js`, `js/supabase-errors.js`	Ícones, formatação, escape de conteúdo e mensagens de erro.
`database/`	Esquema inicial, dados de exemplo, migrações e verificações SQL.
`img/`	Imagens e favicon.
`tests/`	Testes de lógica, segurança e navegador.

Configuração do Supabase

1. Crie ou selecione um projeto Supabase.
2. Copie `.env.example` para `.env` e preencha `VITE_SUPABASE_URL` e `VITE_SUPABASE_PUBLISHABLE_KEY`.
3. Para um banco novo, revise e execute `database/supabase.sql` no SQL Editor.
4. Em uma instalação existente, compare o esquema antes de aplicar alterações. As migrações numeradas são incrementais: aplique `002_area_usuario.sql` e depois `003_cursos_datas_status.sql` quando os fluxos correspondentes forem usados. Não reaplique o SQL inicial sobre tabelas já existentes.
5. Execute os arquivos `database/verificar.sql` e `database/verificar-area-usuario.sql` no SQL Editor para inspecionar a configuração.
6. Configure URL do site e URLs de retorno de autenticação no painel do Supabase, incluindo o endereço de recuperação de senha.

Somente a URL e a chave **publicável** podem estar nas variáveis `VITE_` do frontend. Nunca coloque `service_role`, Secret Key, senha do banco ou outro segredo no navegador ou no repositório. O isolamento dos dados depende das políticas RLS configuradas no banco.

Para detalhes de ativação, migrações, permissões e diagnósticos, consulte INTEGRACAO-SUPABASE.md e AREA-USUARIA.md.

Perfis e acesso

Os perfis usados pela aplicação são `usuario`, `funcionario` e `admin`. O banco é responsável por autorizar cada operação; esconder um botão ou link no frontend não substitui RLS.

- **Visitante:** acessa conteúdo publicado e pode enviar os formulários públicos permitidos.
- **Usuária:** acessa seus próprios dados, consultas, agenda, solicitações, perfil e notificações.
- **Funcionário:** usa as rotinas de atendimento liberadas ao papel de equipe.
- **Administrador:** gerencia os cadastros e as páginas administrativas restritas.

As regras detalhadas por tabela estão no SQL e nos guias de integração. Não altere permissões para resolver um erro sem revisar as políticas e os vínculos de dados.

Cursos, projetos e formulários

O painel compartilha a lógica de CRUD entre as páginas e define suas configurações em `js/admin-schema.js`. Cursos usam a tabela `servicos` por compatibilidade com o esquema existente; o conteúdo exibido ao público é chamado de curso. A migração `003_cursos_datas_status.sql` adiciona datas de início e fim e status. A página principal e a área da usuária leem os cursos publicados/ativos conforme os filtros implementados no catálogo.

As imagens de cursos e projetos podem usar o Storage do Supabase conforme a configuração do painel. Projetos têm status e datas; cursos têm status, datas e indicador de publicação. Solicitações e mensagens ficam no banco e podem estar vinculadas a outros registros: antes de apagar dados, verifique as chaves estrangeiras e o histórico relacionado.

Publicação

O comando `npm run build` gera o conteúdo estático em `dist/`. Os arquivos `vercel.json` e `netlify.toml` definem o build e a pasta de saída para os respectivos provedores. Ao publicar pelo provedor, configure as duas variáveis `VITE_SUPABASE_*` no ambiente de build. Alterações nessas variáveis exigem uma nova compilação.

Após publicar, configure o domínio HTTPS como URL do site no Supabase e permita os redirecionamentos de autenticação usados pela aplicação. Verifique páginas internas abrindo suas URLs diretamente, login, recuperação de senha, permissões e visualização em celular. Não publique `.env`, `node_modules`, arquivos temporários de testes ou chaves secretas.

Documentos relacionados

- README.md: instruções existentes de instalação, banco, operação e publicação.
- AREA-USUARIA.md: ativação e comportamento da área individual.
- INTEGRACAO-SUPABASE.md: integração e diagnóstico do Supabase.

Manual de operação e manutenção

Consulte DOCUMENTACAO.md para uma visão geral das páginas, módulos, configuração, permissões e publicação do projeto. O PDF consolidado pode ser regenerado com `npm run docs:pdf`.

Nova Área da Usuária: consulte AREA-USUARIA.md para ativar o cadastro, agenda, consultas, solicitações, perfil e notificações. A alteração usa a migração incremental `database/migrations/002_area_usuario.sql`; não execute novamente o SQL inicial em um banco existente.

Site público e painel de gestão para uma instituição de acolhimento de mulheres. Feito com HTML5, CSS3, JavaScript modular, Vite e `@supabase/supabase-js`. O banco é PostgreSQL, com Supabase Auth, Storage privado e Row Level Security.

O código completo está nos arquivos deste projeto. Não é preciso copiar trechos desta documentação para montar o site. Os exemplos de SQL para autorizar a primeira conta devem ser adaptados à sua conta real.

Integração atualizada: `.env` foi configurado para o projeto `upleyrxuhdqotaccvcuu` usando somente a chave publicável. O Supabase Auth respondeu HTTP 200, mas as sete tabelas retornaram `PGRST205` na verificação de 11/09/2026. O teste de INSERT também foi recusado porque `mensagens` não está disponível. Execute o SQL da etapa 3 e repita os testes; o banco remoto ainda não está pronto para os cadastros. Veja INTEGRACAO-SUPABASE.md para código completo da configuração, diagnóstico e próximos passos. Sem variáveis, o modo ilustrativo continua disponível; com elas, o site consulta o banco real.

1. Estrutura das pastas e execução local

```
casa-mamae-margarida/
├── index.html                  Página inicial
├── solicitar-atendimento.html Solicitação pública
├── doacoes.html                Doações públicas
├── voluntariado.html           Oportunidades e cadastro de voluntariado
├── contato.html                Contato público
├── login.html                  Login da equipe
├── recuperar-senha.html        Redefinição de senha
├── dashboard.html              Indicadores da equipe
├── beneficiarias.html          Consulta e status das solicitações
├── atendimentos.html           Cadastro, edição e consulta
├── cursos.html                 Gestão de cursos
├── projetos.html               Gestão de projetos
├── gestao-doacoes.html          Gestão privada de doações
├── mensagens.html              Mensagens recebidas
├── relatorios.html             Filtros e exportação CSV
├── usuarios.html               Autorização de perfis
├── configuracoes.html          Uploads e arquivos
├── css/
│   ├── style.css               Site e componentes compartilhados
│   ├── login.css               Login e recuperação
│   └── dashboard.css           Painel responsivo
├── js/
│   ├── supabase.js             Cliente e persistência da sessão
│   ├── layout.js               Cabeçalho, rodapé e marca
│   ├── main.js                 Site, cursos, projetos e diálogos
│   ├── catalog.js              Conteúdo ilustrativo sem credenciais
│   ├── forms.js                Envio dos formulários públicos
│   ├── voluntariado.js         Filtros de vagas e cadastro de voluntariado
│   ├── login.js                Login e recuperação de senha
│   ├── dashboard.js            Consultas, CRUD, relatórios e Storage
│   ├── admin-schema.js         Campos e colunas dos cadastros
│   └── utils.js                Ícones, CPF, datas e escape de texto
├── img/                        Fotografias ilustrativas e favicon
├── database/
│   ├── supabase.sql            Banco, índices, restrições, RLS e bucket
│   └── seed.sql                Catálogo ilustrativo opcional
├── tests/                      Testes de SQL, validação e navegador
├── scripts/create-pages.mjs    Gerador dos HTML secundários
├── .env.example                Modelo de configuração
├── vite.config.js              Compilação de todas as páginas
├── playwright.config.js        Testes desktop e mobile
├── netlify.toml                Configuração de publicação
└── package.json                Dependências e comandos
```

Os cadastros compartilham `dashboard.js` e `admin-schema.js`, em vez de duplicar a lógica de conexão e segurança em um arquivo por entidade. Os HTML permanecem separados e podem ser abertos por suas próprias URLs.

Instale Node.js 22.12+ ou 24 e execute na pasta do projeto:

```
npm.cmd install
npm.cmd run dev
```

No Windows, `npm.cmd` evita o bloqueio do `npm.ps1` pela política de execução do PowerShell. Em macOS/Linux, use `npm`.

Teste: abra o endereço exibido pelo Vite, normalmente `http://localhost:5173`. A página deve exibir a marca, a fotografia, os cursos e os projetos. Não abra o HTML com `file://`, pois os módulos precisam do servidor.

2. Criar o projeto no Supabase

1. Entre no painel do Supabase e crie um projeto na sua organização.
2. Dê um nome ao projeto, escolha a região e guarde a senha do banco em um local seguro.
3. Aguarde o provisionamento.
4. Em Authentication, mantenha login por e-mail habilitado. Para a nova Área da Usuária, aplique a migração 002 e habilite cadastro público com confirmação de e-mail; novas contas recebem somente o perfil `usuario`. A autorização de equipe continua sendo administrativa.
5. Em Authentication → URL Configuration, configure a Site URL local e adicione `http://localhost:5173/recuperar-senha.html` às URLs permitidas. Se usar outro host ou porta, inclua a URL correspondente.

Teste: confirme que você consegue abrir SQL Editor, Authentication e Storage no projeto. Consulte a documentação de URLs de redirecionamento para ajustar os ambientes local e publicado.

3. Criar o banco completo

Abra `database/supabase.sql`, copie todo o conteúdo e execute no SQL Editor de um projeto novo. O arquivo é transacional e deve ser executado uma vez; não apaga nem recria tabelas existentes.

O SQL cria as sete tabelas pedidas, PKs UUID, FKs, valores padrão, índices, validação de CPF, datas, campos de consentimento, RLS e políticas do Storage. Consentimento e sua data foram acrescentados para registrar a autorização do formulário. O CPF não é único: uma mesma mulher pode enviar mais de uma solicitação. Antes de criar atendimentos, a equipe deve conferir se há cadastros duplicados.

Opcionalmente, execute `database/seed.sql` para cadastrar o catálogo ilustrativo. Confirme a disponibilidade com a instituição antes de publicá-lo. Não são inseridas pessoas, doações ou atendimentos fictícios no banco real.

Teste: no Table Editor, confira `perfis`, `beneficiarias`, `atendimentos`, `servicos`, `projetos`, `doacoes` e `mensagens`. Todas devem ter RLS habilitada. No Storage deve existir o bucket privado `casa-mamae-margarida`.

4. Configurar `js/supabase.js`

Copie `.env.example` para `.env`:

```
Copy-Item .env.example .env
```

Preencha com os dados do seu projeto:

```
VITE_SUPABASE_URL=https://SEU-PROJETO.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=<configure in .env>
```

No painel, o diálogo **Connect** mostra a URL e a chave publicável. As chaves também estão na área de API Keys das configurações do projeto. A integração usa `VITE_SUPABASE_PUBLISHABLE_KEY`, com uma chave `sb_publishable_...`. Consulte API keys.

Nunca use `service_role`, `sb_secret_...` nem a senha do banco no frontend. Variáveis `VITE_` são incorporadas ao JavaScript entregue ao navegador: não são segredos. É a RLS que protege os registros.

O arquivo `js/supabase.js` importa `createClient` de `@supabase/supabase-js`, lê essas variáveis e configura a sessão. Reinicie o Vite após alterar `.env`.

Teste: os cursos devem ser carregados do seu banco. Um banco vazio mostra estado vazio, e um erro de conexão mostra uma mensagem, sem substituir dados reais por dados ilustrativos.

5. Página inicial

`index.html`, `css/style.css`, `js/layout.js` e `js/main.js` implementam navegação, banner, apresentação, missão, visão, valores, cursos, projetos, convite para doação, contato e rodapé. O layout usa rosa queimado, bege, branco e dourado suave.

A flor da marca é um espaço reservado para o logotipo oficial. Substitua o símbolo no componente `brand` de `js/layout.js`. Atualize também `img/favicon.svg` quando houver um ícone oficial.

Teste: clique nos itens do menu, abra os detalhes de um curso, alterne missão/visão/valores e abra um projeto. Em uma tela estreita, teste o botão do menu. Use Tab, Enter e Escape nos diálogos.

6. Login

`login.html` usa `supabase.auth.signInWithPassword()`. Há mostrar/ocultar senha, recuperação e opção de manter a sessão.

Em Authentication → Users, crie a primeira conta da equipe e confirme o e-mail conforme o procedimento do seu projeto. Em seguida, autorize esse usuário com SQL, substituindo o endereço abaixo pelo e-mail da conta criada:

```
insert into public.perfis (user_id, nome, email, cargo, role)
select id, 'Nome da administradora', email, 'Administração', 'admin'
from auth.users
where email = 'SEU-EMAIL-REAL@EXEMPLO.COM'
on conflict (user_id) do update set role = 'admin', cargo = excluded.cargo;
```

Se nenhum registro for inserido, o e-mail não foi encontrado em Auth. Não cadastre a senha na tabela `perfis`.

Teste: entre com a conta autorizada. Uma conta Auth sem perfil deve receber “acesso ainda não autorizado”; credenciais erradas não devem liberar o painel.

7. Autenticação, recuperação e perfis

Antes de buscar dados administrativos, o painel valida o usuário com `auth.getUser()` e consulta seu perfil. O banco confirma as permissões em cada operação. Funcionários não acessam doações, relatórios, usuários, mensagens ou configurações.

“Manter sessão ativa” usa a persistência do Supabase em `localStorage`; sem essa opção, a sessão usa `sessionStorage`. Nenhuma senha é armazenada manualmente. Ao sair, a sessão é removida e a página retorna ao login.

“Esqueci minha senha” envia uma solicitação via `resetPasswordForEmail`. O link abre `recuperar-senha.html`, que chama `updateUser` para salvar a nova senha. Ajuste SMTP e remetente do projeto para o envio real; o serviço padrão do Supabase tem restrições. Veja autenticação por senha.

Teste: faça login, recarregue, saia e tente abrir `beneficiarias.html` diretamente. Depois, solicite um link de recuperação, abra o e-mail e redefina a senha. Sem sessão válida, a página de recuperação deve recusar a alteração.

8. Dashboard

`dashboard.html` mostra seis indicadores reais e gráficos de barras com contagens textuais: atendimentos por mês, tipos solicitados e solicitações por status. A leitura percorre as páginas da API para evitar estatísticas cortadas pelo limite de resposta.

Teste: com o banco vazio, os totais devem ser zero. Cadastre uma solicitação pública e confira o aumento de beneficiárias e pendências. Não existem percentuais de crescimento ou números de impacto inventados.

9. Cadastro de beneficiárias

`solicitar-atendimento.html` coleta os campos solicitados e exige autorização de uso dos dados. O CPF passa por validação no navegador e no banco. A API pública pode inserir apenas os campos do formulário, sem escolher status, ID ou datas internas, e não pode ler os cadastros.

`beneficiarias.html` permite pesquisa, consulta detalhada, atualização de status e histórico de atendimentos. Funcionários podem alterar somente o status; o banco bloqueia alterações de identidade feitas por esse perfil. Excluir um cadastro é uma ação administrativa e exige confirmação. Cadastros com atendimentos vinculados são protegidos por FK.

Teste: use somente dados de teste autorizados. Envie um formulário, confira o status inicial “Pendente” e consulte-o como equipe. Em janela anônima, a tabela de beneficiárias não deve ser legível via API.

10. Sistema de atendimentos

Em `atendimentos.html`, crie um atendimento, escolha uma beneficiária, informe tipo, data, hora, descrição e status. O responsável inicial é o usuário autenticado. Edição não permite trocar os vínculos do atendimento. O responsável referencia a conta Auth, de modo que revogar seu perfil não apaga a identificação histórica da conta.

Uma beneficiária pode ter vários atendimentos. O histórico da beneficiária mostra os encontros registrados, com data, situação, descrição e observação; não é um log de cada edição. Status da solicitação e de seus atendimentos são independentes, para acomodar vários acompanhamentos simultâneos.

Teste: crie dois atendimentos para o mesmo cadastro, finalize um deles e abra o histórico na tela de beneficiárias. Confirme que os dois aparecem e que o painel atualiza suas contagens.

11. Cursos

Administradores cadastram, editam, ativam/desativam e excluem cursos em `cursos.html`. Também informam início, fim e status da turma. Em bancos existentes, execute `database/migrations/003_cursos_datas_status.sql` no SQL Editor do Supabase antes de usar esses campos. Para compatibilidade com os dados existentes, os cursos continuam armazenados na tabela `servicos` e as imagens usam a pasta `servicos/`.

Teste: cadastre um curso e confira o site. Desative-o e recarregue: ele deve desaparecer do catálogo público, mas continuar visível para a equipe.

12. Projetos e Storage

Em `projetos.html`, o administrador define nome, descrição, período, imagem e status. A data final não pode ser anterior à inicial. Somente projetos com status “Ativo” aparecem no site.

O bucket é **privado**, inclusive para imagens. As políticas liberam leitura apenas das pastas editoriais `servicos/`, `projetos/` e `institucional/`; a aplicação gera URLs assinadas para exibi-las. A pasta `documentos/` permite leitura apenas para a equipe autorizada. Uploads são administrativos; PDF é aceito apenas em documentos. O tamanho máximo é 10 MB.

Configurações permite enviar e listar arquivos. As URLs de consulta de documentos expiram em 60 segundos. Nomes são UUIDs para não expor nomes de pessoas nas URLs. Não envie fotos de documentos pessoais para pastas editoriais: imagens nessas pastas têm leitura pública. A instituição deve definir critérios de autorização e retenção dos arquivos.

Teste: envie uma imagem de projeto e confira o card público. Envie um PDF autorizado à pasta privada e tente consultar seu caminho sem sessão: o acesso deve ser recusado. Confirme as políticas no Supabase real; os testes locais não executam o serviço HTTP do Storage.

13. Doações e contato

`doacoes.html` apresenta Pix, alimentos, roupas, higiene, materiais escolares e voluntariado. O formulário registra uma **intenção de doação**, não processa pagamentos. Nenhuma chave Pix fictícia é apresentada.

O administrador acompanha o status em `gestao-doacoes.html`. Mensagens de `contato.html` aparecem em `mensagens.html`, com leitura e marcação de lida. Nenhuma mensagem é enviada automaticamente a terceiros por e-mail.

Teste: registre uma intenção de doação e uma mensagem de teste; verifique ambas com a conta administrativa. A conta de funcionário e o visitante não devem poder consultar essas informações.

14. Relatórios

`relatorios.html` oferece filtros de período, status e tipo de atendimento, indicadores, gráficos e download CSV agregado. Solicitações usam sua data de cadastro; atendimentos usam a data do encontro. Projetos, cursos e doações usam a data de cadastro e não o status do atendimento. “Mulheres com atendimento” conta IDs distintos de beneficiárias com encontros em andamento ou finalizados; possíveis cadastros duplicados precisam ser resolvidos pela equipe.

Teste: crie encontros em dois meses diferentes, filtre um deles e compare as contagens. Informe um período invertido para conferir a validação. O CSV deve incluir os filtros e os indicadores, sem nomes, CPFs ou endereços.

15. Row Level Security e tratamento dos dados

Recurso	Visitante	Funcionário	Administrador
Cursos/projetos ativos	Ler	Ler todos	Gerenciar
Beneficiárias	Enviar solicitação	Consultar e atualizar status	Consultar, atualizar status e excluir
Atendimentos	Sem acesso	Criar, consultar e atualizar	Gerenciar
Doações/mensagens	Enviar formulário	Sem leitura	Gerenciar
Perfis	Sem acesso	Ler o próprio	Autorizar e gerenciar outros
Documentos	Sem acesso	Ler	Ler e enviar
Imagens editoriais	Ler	Ler	Ler e enviar

A autorização vem da tabela `perfis`, nunca de metadados definidos pelo visitante. A função `equipe_role()` tem `search_path` fixo e evita recursão da política de perfis. Não há política para autoatribuição de privilégios. O administrador não pode excluir/rebaixar o próprio perfil pela interface.

Textos dos registros são escapados antes de inserir HTML. Formulários têm limites de tamanho e restrições no banco. Consentimento é registrado no servidor. A aplicação não grava dados pessoais em cache local próprio nem exporta essas informações nos relatórios.

Antes de operar com dados reais, a instituição deve preencher seu aviso de privacidade com responsável, finalidade, base legal aplicável, retenção e canal de direitos. O texto da interface é um ponto de partida, não uma declaração de conformidade jurídica. O envio público direto à API tem um honeypot básico, mas não tem rate limiting próprio: para operação exposta a abuso, adote um endpoint/Edge Function com limites e CAPTCHA validado no servidor, revogando o INSERT público direto. Não trate um CAPTCHA apenas visual como proteção da API.

Teste: execute os testes SQL abaixo e repita os cenários em um projeto de homologação. Nunca desative RLS para corrigir uma mensagem de permissão; verifique a política, o perfil e os campos enviados.

16. Testes

```
npm.cmd test
npx.cmd playwright install chromium
npm.cmd run test:e2e
npm.cmd run build
npm.cmd run preview
```

`npm test` executa a migração real em PostgreSQL via PGlite, com esquemas de Auth/Storage simulados para testar permissões. Cobre inserção pública, privacidade, elevação de privilégio, status, CPF, consentimento, relacionamentos e documentos privados. Apenas a extensão `pgcrypto` é omitida nesse ambiente, porque `gen_random_uuid` já é nativo.

Os testes Playwright públicos simulam o cliente sem configuração, mesmo com `.env` presente, e nunca enviam seus formulários ao banco real. Eles verificam navegação, catálogo, diálogos, login e responsividade. Os testes administrativos usam um cliente simulado; as políticas são exercitadas separadamente em PostgreSQL local. Use `npm.cmd run test:supabase` para um diagnóstico de leitura contra o projeto real e adicione `-- --insert-test` para criar uma mensagem sintética identificada. Login com credenciais de equipe, recuperação por e-mail e Storage ainda precisam de validação após a criação do banco e da primeira conta autorizada.

Teste: os comandos devem terminar sem falhas. Abra o preview, teste os links e confira os estados vazios. Teste também o fluxo completo como admin, funcionário e visitante após conectar o Supabase.

17. Publicação

1. Confirme textos, logo, cursos disponíveis, imagens autorizadas, contatos, programação e informações de doação com a instituição.
2. Configure o Supabase e valide o fluxo completo em homologação.
3. Execute `npm.cmd run build`. O resultado estará em `dist/`, com todas as páginas HTML.
4. Em uma hospedagem estática, publique a pasta `dist`. No Netlify, o arquivo `netlify.toml` já define `npm run build`, `dist` e cabeçalhos básicos. Veja Vite no Netlify.
5. Se a hospedagem fizer o build, adicione `VITE_SUPABASE_URL` e `VITE_SUPABASE_PUBLISHABLE_KEY` nas variáveis de ambiente antes de compilar. Alterá-las exige novo build.
6. Atualize a Site URL do Supabase para o domínio HTTPS e permita `https://SEU-DOMINIO/recuperar-senha.html` como redirecionamento. Configure SMTP para e-mails reais.
7. Não publique `node_modules`, `.env` nem pastas de resultados de teste. Não configure um redirecionamento geral para `index.html`: este é um site com múltiplas páginas.

Teste: no domínio final, abra `/login.html` e `/solicitar-atendimento.html` diretamente, recarregue as páginas, envie um registro de teste autorizado e valide recuperação por e-mail. Verifique a página no celular e o bloqueio de acesso administrativo sem sessão.

Imagens e manutenção

As fotografias são ilustrativas, obtidas do Unsplash, e não representam beneficiárias ou atividades comprovadas da instituição. Identificadores das imagens: `photo-1529156069898-49953e39b3ac`, `photo-1455390582262-044cdead277a`, `photo-1511632765486-a01980e01a18` e `photo-1517486808906-6ca8b3f04846`. A marca usa uma flor do Lucide como elemento provisório. As fontes são DM Sans e Lora, com fontes locais de fallback.

O gerador `scripts/create-pages.mjs` serviu para criar os HTML secundários. Você pode editá-los diretamente; executar novamente o gerador sobrescreve essas páginas, portanto atualize também o template caso queira continuar usando essa ferramenta.

Área da usuária

A implementação acrescenta cadastro público, consultas, agenda, solicitações, histórico, perfil e notificações. O site público e o painel administrativo existentes continuam no mesmo projeto e usam o mesmo `js/supabase.js`.

Para ativar no seu Supabase e na Vercel

1. Abra o SQL Editor do projeto `upleyrxuhdqotaccvcuu`.
2. Execute **somente** database/migrations/002_area_usuario.sql, uma vez, sobre o banco existente. **Não execute novamente** `database/supabase.sql`. A migração é transacional e não remove registros.
3. Execute database/verificar-area-usuario.sql. Confira as colunas, as três tabelas novas, políticas, funções e trigger de cadastro.
4. Em Supabase → Authentication, habilite cadastro de novas contas por e-mail e mantenha a confirmação de e-mail ativada. Configure o remetente/SMTP para o envio real de mensagens de confirmação e recuperação.
5. Em Authentication → URL Configuration, defina a **Site URL** como `https://prjtmargarida.vercel.app` e permita estas URLs exatas:

``` text https://prjtmargarida.vercel.app/area-usuario.html https://prjtmargarida.vercel.app/redefinir-senha.html https://prjtmargarida.vercel.app/recuperar-senha.html ```

1. Na Vercel, mantenha `VITE_SUPABASE_URL` e `VITE_SUPABASE_PUBLISHABLE_KEY`. O projeto tem `vercel.json` com build `npm run build` e saída `dist`. Publique uma nova versão contendo os arquivos desta alteração, **após** aplicar a migração. Não configure um rewrite geral de todas as URLs para `index.html`.
2. Abra `https://prjtmargarida.vercel.app/cadastro.html`, crie uma conta de teste autorizada e confirme o e-mail. Ao entrar, uma conta `usuario` segue para `area-usuario.html`; `admin` e `funcionario` seguem para o dashboard existente.

Os links de e-mail usam a URL de produção informada, mesmo quando o formulário é aberto localmente. A configuração está em js/auth-routing.js. A confirmação precisa encontrar o código novo já publicado na Vercel. A página antiga `recuperar-senha.html` foi preservada para links anteriores.

O formulário verifica `cadastro_disponivel()` antes de chamar `signUp`, evitando que este frontend crie uma conta antes da migração. Nenhuma chave secreta é necessária. A configuração de redirects segue a documentação de URLs do Supabase.

Limite desta entrega: o código e a migração foram implementados e testados localmente. Não foi executada a migração no seu projeto remoto, nem feito deploy na sua conta Vercel. A chave publicável disponível permite usar a API, mas não administrar o esquema do banco. Não foram criadas contas ou enviados e-mails reais durante os testes.
```

O que foi reutilizado

Recurso existente	Complemento
`perfis`	Aceita `usuario` e recebe nascimento, CPF, telefone e caminho da foto.
`beneficiarias`	Recebe `usuario_id` opcional e único para vincular a nova conta. Registros antigos permanecem sem vínculo automático.
`atendimentos`	Continua sendo a única tabela de consultas; recebe profissional, solicitação de origem, local e orientação para a usuária.
`servicos`	Alimenta o novo formulário de solicitação.
Supabase Auth	Mesmo cliente: `signUp`, login, recuperação, sessão e logout.
Storage	Mesmo bucket privado; nova pasta `avatares/UUID-DA-CONTA/`.
Dashboard	Novos itens Solicitações das usuárias e Profissionais; agendamento usa o formulário existente.

As tabelas novas são `solicitacoes`, `notificacoes` e `profissionais`. Seus endpoints não existiam na API na análise inicial. Não foi criada uma tabela duplicada de consultas ou de usuárias.

O vínculo é criado por trigger no banco, na mesma transação do cadastro Auth. A trigger fixa `role = 'usuario'` e ignora qualquer `role` enviado em metadados. Nascimento, CPF, telefone e consentimento são validados também no banco. Não há escolha pública de perfil nem senha em uma tabela da aplicação. O padrão usa o fluxo de gerenciamento de usuários do Supabase.

Para cadastrar uma conta sem inventar endereço ou pedido de atendimento, endereço, bairro, cidade, motivo e tipo de atendimento passam a aceitar `NULL` em `beneficiarias`. A política do formulário público antigo continua exigindo seus campos originais. Cadastros anteriores não são alterados. O telefone editado no perfil é sincronizado com a beneficiária vinculada.

Fluxo da usuária

1. **Cadastro:** informa dados pessoais, senha, confirmação e consentimento. Recebe a orientação para confirmar o e-mail. Contas duplicadas recebem uma mensagem neutra, sem expor a existência da conta.
2. **Login:** o perfil é consultado no banco para decidir o destino. As páginas protegidas validam o usuário com `auth.getUser()`; a RLS e as funções verificam a identidade no servidor.
3. **Início:** mostra próxima consulta, consultas realizadas, solicitações pendentes e próximos compromissos. Quando não há dados, exibe estados vazios.
4. **Minhas consultas:** exibe atendimento, profissional, data, horário, local, situação e orientação liberada à usuária.
5. **Minha agenda:** organiza compromissos futuros em Hoje, Próximos dias, Este mês e Mais adiante. As categorias não repetem compromissos. Canceladas e realizadas não entram na agenda futura.
6. **Solicitar atendimento:** escolhe serviço ativo e informa motivo, data, horário e observações. A preferência não é uma confirmação de consulta.
7. **Minhas solicitações:** acompanha serviço, data do pedido, preferência e status.

8. **Histórico:** exibe somente consultas realizadas.
9. **Meu perfil:** permite telefone e foto; nome, CPF, nascimento e e-mail são somente leitura. Correções de identidade são tratadas com a equipe. A foto fica privada e usa URL temporária.
10. **Notificações:** apresenta novidades persistidas no banco e permite marcar como lidas. O contador indica apenas as não lidas.

Os dados e o contador são atualizados a cada **30 segundos** com a aba visível, ao retornar à aba e pelo botão Atualizar. Não depende de habilitar Realtime no Supabase. Os formulários abertos não são recriados durante a atualização, preservando o que a pessoa está digitando. As notificações são mensagens dentro do site; não são push nem e-mails automáticos.

Fluxo da equipe no painel existente

1. Em **Profissionais**, o administrador cadastra nome e especialidade. Nenhuma pessoa foi cadastrada automaticamente. Um profissional pode atuar sem ter uma conta Auth; o responsável interno pelo registro continua sendo `funcionario_id`.
2. Em **Solicitações das usuárias**, a equipe consulta os pedidos, abre detalhes e muda o status pelo botão Editar. Somente o status pode ser alterado depois do envio.
3. Ao definir **Aprovada**, aparece o botão **Agendar atendimento** naquela linha.
4. Esse botão abre o formulário de atendimentos existente, com a beneficiária, serviço e preferência preenchidos. A equipe confirma a data/hora, escolhe o profissional, informa local e registra a descrição interna. A validação do banco impede vincular uma solicitação a outra beneficiária.
5. O campo **Orientação visível à usuária** pode conter informações de preparação para o encontro. **Descrição interna** e **Observação interna** continuam privadas.
6. Para confirmar, reagendar, cancelar ou realizar, abra o atendimento e altere seu status. Para reagendar, altere também data/hora.
7. Ao marcar como **Realizada** ou pelo status antigo **Finalizado**, a solicitação vinculada passa a **Concluída**. A usuária recebe uma notificação de atualização e vê o encontro no histórico.

Uma solicitação aprovada pode originar mais de um atendimento, permitindo acompanhamento. Finalizar um desses atendimentos conclui o pedido de agendamento; não exclui outros encontros. Solicitações não têm exclusão pela interface, preservando sua vinculação e acompanhamento.

Os status antigos continuam válidos para preservar registros e fluxos anteriores. Na leitura individual:

Status anterior	Exibição para a usuária
`Pendente` / `Em análise`	Agendada
`Em atendimento`	Confirmada
`Finalizado`	Realizada

Novos atendimentos podem usar diretamente Agendada, Confirmada, Realizada, Cancelada e Reagendada. Os relatórios administrativos passam a incluir Realizada nas contagens de conclusão, mantendo Finalizado.

Privacidade e políticas

RLS controla linhas; liberar a linha completa de um atendimento também liberaria notas internas. Por isso, a usuária **não recebe SELECT direto** de `atendimentos` ou `beneficiarias`. Sua agenda, consultas e histórico usam `minhas_consultas()`, que retorna apenas os campos permitidos e filtra pelo vínculo com `auth.uid()`. Essa função não aceita um ID de outra pessoa como parâmetro. `minhas_solicitacoes()` segue o mesmo padrão, preservando o nome do serviço mesmo se ele for desativado depois.

Recurso	Acesso da usuária
Perfil	Ler o próprio; alterar telefone/foto, com trigger que bloqueia identidade e permissões.
Consultas e histórico	Função com projeção segura dos próprios atendimentos; nunca notas internas.
Solicitações	Inserir para si com status Pendente; ler as próprias; sem alterar status.
Notificações	Ler as próprias; atualizar somente `lida`; sem criar mensagens falsas.
Profissionais	Sem leitura direta do cadastro; apenas o nome associado à própria consulta.
Avatares	Upload, leitura e remoção apenas no diretório do próprio UUID.
Documentos da equipe	Sem acesso.

As funções com privilégios elevados têm `search_path` fixo e permissões explícitas. As políticas antigas de admin/funcionário permanecem. O papel público não pode se promover via formulário, atualização de perfil, metadados ou troca de ID no navegador.

Históricos anteriores: não são associados automaticamente pelo CPF nem por e-mail. Isso poderia liberar um histórico sensível para uma pessoa que informou dados de outra. Contas novas ganham um novo registro vinculado. Se houver duplicidade com um cadastro anterior, a administração deve verificar a identidade e tratar os vínculos de forma controlada no banco. A migração não toma essa decisão sozinha.

Arquivos e código completo

O código completo está nos arquivos do projeto, pronto para revisão. Não é necessário copiar fragmentos deste guia para montar as páginas.

Novas páginas: `cadastro.html`, `area-usuario.html`, `minhas-consultas.html`, `agenda.html`, `minhas-solicitacoes.html`, `nova-solicitacao.html`, `historico.html`, `perfil.html`, `notificacoes.html`, `redefinir-senha.html`, `solicitacoes.html` e `profissionais.html`.

Novos módulos: js/cadastro.js, js/usuario.js, js/usuario-data.js e js/auth-routing.js. O estilo complementar está em css/usuario.css.

Banco: migração completa e SQL de verificação.

Arquivos existentes alterados: `login.html`, `js/login.js`, `js/dashboard.js`, `js/admin-schema.js`, `js/supabase-errors.js`, `vite.config.js`, testes administrativos e documentação. `js/supabase.js`, as credenciais, o SQL inicial e os dados existentes foram preservados.

Configuração e apoio: `vercel.json` e `scripts/create-user-pages.mjs`. O gerador cria somente as páginas novas; executar novamente sobrescreve essas páginas. Prefira editar os arquivos entregues e manter o template sincronizado caso escolha usar o gerador.

Testes

```
npm.cmd test
npm.cmd run test:e2e
npm.cmd run build
npm.cmd run dev
```

Os testes SQL executam o banco inicial e a nova migração em PostgreSQL via PGlite. Criam duas usuárias fictícias somente no banco local de testes e verificam criação atômica, preservação da equipe, isolamento, ausência de notas internas, proteção de perfil, vínculo de solicitações, notificações e avatares.

Os testes de navegador usam um cliente simulado, sem cadastrar pessoas, inserir dados ou enviar e-mails no Supabase real. Verificam o cadastro, redirecionamento por perfil, solicitação, edição de telefone, marcação de notificações, atualização automática, layout e agendamento pelo painel existente, em computador e celular.

Após publicar, faça também a validação de ponta a ponta com duas contas de teste autorizadas:

1. Cadastre A e B, confirme os e-mails e confira `role = usuario` no banco.
2. A solicita um serviço. B não deve ver o pedido.
3. Como equipe, aprove o pedido de A e agende. A deve ver a consulta; B não.
4. Reagende e aguarde até 30 segundos com a aba de A aberta. Confira data/status e notificação.
5. Preencha uma nota interna e uma orientação pública diferentes. Somente a orientação deve aparecer para A.
6. Finalize e confira histórico e solicitação concluída.
7. Atualize telefone e foto de A. Confira o telefone sincronizado e que B não lê o avatar de A.
8. Solicite recuperação de senha e confira que o e-mail abre `https://prjtmargarida.vercel.app/redefinir-senha.html`.
9. Saia e tente abrir as páginas individuais diretamente: deve voltar ao login. Uma usuária também não deve acessar o dashboard da equipe.

Se aparecer erro de schema/função, confira se a migração foi aplicada inteira. Não desative RLS para contornar permissões. Se o signup retornar erro de banco, confira os logs Auth e a trigger de cadastro; uma falha de validação nessa trigger reverte a criação da conta.

Integração com o Supabase

Complemento mais recente: a Área da Usuária está documentada em AREA-USUARIA.md. Depois de aplicar a migração 002, contas novas recebem `usuario`; para autorizar a equipe, edite o perfil existente. As instruções abaixo registram a integração inicial.

Resultado e impedimento identificado

Arquitetura: **HTML/CSS/JavaScript com Vite**, sem Express ou backend próprio. A dependência `@supabase/supabase-js` já estava instalada. O design e os HTML/CSS foram preservados.

O `.env` local agora aponta para `upleyrxuhdqotaccvcuu`. Em 11/09/2026, `/auth/v1/settings` respondeu **HTTP 200**, com login por e-mail habilitado. Isso confirma que URL e chave publicável são aceitas; não equivale a um login de usuário.

Os testes GET de `perfis`, `beneficiarias`, `servicos`, `atendimentos`, `projetos`, `doacoes` e `mensagens` retornaram **PGRST205 — tabela não encontrada no cache do schema**. SELECT em `servicos` e INSERT em `mensagens` também retornaram esse erro. **Nenhuma mensagem de teste foi criada.** Não foi possível inspecionar o esquema remoto ou comprovar suas políticas pela chave publicável. A migração existente é compatível com o código e foi testada em PostgreSQL local.

Para completar a integração remota, abra o SQL Editor deste projeto, confira se as tabelas já existem no Table Editor e, se ainda não existem, execute **todo** o arquivo database/supabase.sql. Ele cria tabelas, relacionamentos, índices, consentimento, RLS e Storage. Não execute essa migração de criação sobre tabelas preexistentes: se elas existirem, compare seu esquema com o código e investigue exposição/cache da API antes de alterar algo.

Depois execute database/verificar.sql, que faz consultas administrativas de verificação sem ler dados de beneficiárias. Deve mostrar sete tabelas com RLS ativa, as políticas e o bucket privado. A chave publicável não pode executar SQL administrativo. Não é necessário enviar uma Secret Key para esta integração.

Arquivos alterados e criados

Alterados: `js/supabase.js`, `.env.example`, `.gitignore`, `js/forms.js`, `js/main.js`, `js/login.js`, `js/dashboard.js`, `package.json`, `tests/browser/public.spec.js` e `README.md`.

Criados: `.env` (ignorado pelo Git), `js/supabase-errors.js`, `scripts/test-supabase.mjs`, `database/verificar.sql`, `tests/browser/connection.spec.js` e este guia.

O cliente compartilhado passou a ler a variável `VITE_SUPABASE_PUBLISHABLE_KEY`. Foram acrescentadas mensagens específicas para tabela/coluna ausente, permissão, campos inválidos e vínculos. O formulário de atendimento não oferece serviços ilustrativos quando o banco real falha. Os testes de interface foram isolados para não enviar registros reais por causa do novo `.env`.

Configuração local

Arquivo `.env` já preenchido:

```
VITE_SUPABASE_URL=https://upleyrxuhdqotaccvcuu.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=<configure in .env>
```

Esses dois valores são públicos por definição; o acesso aos registros depende das políticas e da sessão. Veja chaves de API do Supabase.

Arquivo `.env.example`:

```
# Frontend Vite: somente URL e chave publicável.
VITE_SUPABASE_URL=
VITE_SUPABASE_PUBLISHABLE_KEY=<configure in .env>

# Reservadas para um futuro backend; não utilizadas neste site estático.
# Nunca adicione VITE_ a uma variável secreta.
SUPABASE_URL=
SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SECRET_KEY=
SUPABASE_JWKS_URL=
```

Arquivo `.gitignore`:

```
node_modules/
dist/
.env
.env.local
.env.*
!.env.example
test-results/
playwright-report/
```

Nenhuma Secret Key foi fornecida, gravada ou incorporada ao navegador. Os campos de backend no exemplo estão vazios e não são usados. O JWKS não é necessário: a aplicação usa Supabase Auth e o painel confirma o usuário com `getUser()`.

Código completo de `js/supabase.js`

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseKey = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY;
export const configured = Boolean(supabaseUrl && supabaseKey);
// Only a publishable/anon key belongs in this file. Access is enforced by RLS.
const storage = {
  getItem: (key) => localStorage.getItem(key) ?? sessionStorage.getItem(key),
  setItem: (key, value) => {
    const persistent = localStorage.getItem('cmm-remember') === 'true';
    (persistent ? sessionStorage : localStorage).removeItem(key);
    (persistent ? localStorage : sessionStorage).setItem(key, value);
  },
  removeItem: (key) => {
    localStorage.removeItem(key);
    sessionStorage.removeItem(key);
  },
};
export const supabase = configured
  ? createClient(supabaseUrl, supabaseKey, {
      auth: { persistSession: true, autoRefreshToken: true, detectSessionInUrl: true, storage },
    })
  : null;
export function requireConnection() {
  if (!supabase)
    throw new Error(
      'O envio ainda não está disponível. A instituição precisa configurar a conexão com o Supabase.
Nenhum dado foi enviado.'
    );
  return supabase;
}
```

O modo sem credenciais foi preservado para visualização e testes. Na configuração atual, o cliente existe e consulta o projeto real. Há um único cliente no frontend; o script de terminal tem seu próprio cliente sem persistência, pois roda em outro processo.

SELECT e INSERT: execução e resultado

O código completo está em `scripts/test-supabase.mjs`. Execute no terminal, a partir da pasta do projeto:

```
npm.cmd run test:supabase
```

Esse comando faz um SELECT de até cinco serviços e verifica os outros endpoints com `limit(0)`, sem baixar dados pessoais. O terminal mostra o nome da operação, o código do erro ou o sucesso. Uma resposta vazia não comprova que a RLS está correta; use também o SQL de verificação e contas com diferentes perfis.

Para testar escrita real após criar as tabelas:

```
npm.cmd run test:supabase -- --insert-test
```

Além das leituras, o script insere uma mensagem com assunto `TESTE TECNICO CMM` e data/hora. Usa somente dados sintéticos e não envia e-mail. A resposta de INSERT não solicita leitura do registro, pois visitantes não devem ler mensagens. Consulte o registro como administrador e depois exclua-o na tela Mensagens. Cada execução bem-sucedida cria um novo registro identificado.

Erros: `PGRST205` indica tabela indisponível; `PGRST204` pode indicar coluna ausente; `42501` indica permissão; `23514` indica restrição de dados. Corrija esquema, GRANT ou política correspondente; não desative RLS. Falhas de rede, chave inválida e erros de backend também aparecem no terminal. O script termina com código 1 quando há falha.

Login, sessão, logout e primeira conta

1. Em Authentication → Users, crie sua conta de equipe.
2. Após executar a migração, autorize a conta no SQL Editor, substituindo o e-mail:

```
insert into public.perfis (user_id, nome, email, cargo, role)
select id, 'Administração da Casa', email, 'Administração', 'admin'
from auth.users
where email = 'SEU-EMAIL-REAL@EXEMPLO.COM'
on conflict (user_id) do update set role = 'admin', cargo = excluded.cargo;
```

1. Reinicie o Vite com `npm.cmd run dev` e abra `login.html`.
2. Em `js/login.js`, o formulário executa:

```
const { data, error } = await client.auth.signInWithPassword({
  email: form.elements.email.value.trim(),
  password: form.elements.password.value,
});
```

O login consulta o perfil antes de liberar a navegação. Em `js/dashboard.js`, `supabase.auth.getUser()` confirma o usuário no serviço Auth antes de consultar registros. Não confiamos somente em uma sessão armazenada localmente. A página de recuperação usa `const { data: { session } } = await client.auth.getSession()` antes de atualizar a senha. O botão Sair chama `await supabase.auth.signOut()` e retorna ao login.

Teste: entre, recarregue a página, saia e abra `dashboard.html` diretamente. Sem sessão, ele deve voltar ao login. Uma conta Auth sem perfil não deve acessar registros. Crie também um funcionário e confirme que as telas administrativas exclusivas ficam bloqueadas.

Para recuperação por e-mail, configure Site URL e permita `http://127.0.0.1:5173/recuperar-senha.html` e/ou `http://localhost:5173/recuperar-senha.html`, conforme o endereço usado. Configure também o domínio final e o envio de e-mails no Supabase. Não foi realizado login real nem envio de recuperação, pois não foram fornecidas credenciais de equipe.

Como testar as páginas já integradas

Área	Arquivos/lógica	Teste após preparar o banco
Serviços	`main.js`, `dashboard.js`, `admin-schema.js`	Como admin, criar, editar, desativar e excluir um serviço; conferir o catálogo público.
Projetos	Mesmos módulos	Cadastrar período e imagem; conferir que apenas projetos ativos são publicados.
Beneficiárias	`forms.js`, `dashboard.js`	Solicitar atendimento com dados de teste autorizados; consultar como equipe e atualizar status.
Atendimentos	`dashboard.js`	Criar dois encontros, editar um, finalizar e conferir o histórico da beneficiária.
Doações	`forms.js`, `dashboard.js`	Registrar intenção e atualizar seu status como administrador.
Contato	Mesmos módulos	Enviar uma mensagem sintética, marcá-la como lida e excluir após o teste.
Dashboard	`dashboard.js`	Conferir contagens com os registros criados; nenhum indicador usa números ilustrativos.
Relatórios	`dashboard.js`	Filtrar data/status/tipo e exportar CSV agregado.
Storage	`dashboard.js`, `main.js`	Enviar imagem como admin; verificar que documentos não podem ser consultados como visitante.

As operações SELECT/INSERT/UPDATE/DELETE e os controles de perfil já existiam; a configuração agora aponta esses fluxos para o projeto informado. Não foi recriado o site nem adicionado backend.

Políticas necessárias e validação

`database/supabase.sql` mantém visitantes com leitura apenas do catálogo ativo e INSERT limitado nos formulários; funcionários podem consultar beneficiárias e atualizar status/atendimentos; administradores gerenciam os cadastros. Perfis não permitem autoelevação, e documentos pessoais ficam em pasta privada. Consulte RLS no Supabase.

```
npm.cmd test
npm.cmd run test:e2e
npm.cmd run build
```

Validação desta alteração: nove testes locais de segurança/validação e 16 testes de navegador passaram, e o build de produção foi gerado. Também foi verificado no navegador, usando o cliente real, que a página mostra a configuração de banco pendente e que o dashboard sem sessão retorna ao login.

Os testes locais não substituem o teste de integração remota. A próxima ação necessária é executar/verificar o SQL no painel; depois repetir SELECT/INSERT e validar login, permissões e Storage com contas reais de teste.